

Министерство науки и высшего образования Российской Федерации
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Образовательная программа: Компьютерные системы и технологии



Отчет

По предмету

Программирование на C++

Лабораторная работа № 4

Студенты:

Андрейченко Леонид Вадимович

Мамонтов Иван Олегович

Преподаватель:

Лаздин Артур Вячеславович

Санкт-Петербург

2024

Задание

В том случае, когда подавляюще большое количество элементов вектора или матрицы равны нулям для их хранения имеет смысл использовать структуры данных, позволяющие существенно экономить память, за счёт того, что нули можно не хранить.

Задание: разработать шаблонные классы для хранения разреженных вектора и 2D матрицы для чего:

- Предложить структуры данных для хранения значений исходных векторов и матриц. (подсказка: использование хеш-таблиц — хорошая идея), рассмотрите необходимость поддержки собственных итераторов;
- Реализовать набор унарных и бинарных операций для вектора и матриц; (транспонирование, сложение, произведение векторов (вспоминаем линейную алгебру), обращение матрицы, возведение матрицы в степень — предусмотреть два варианта с целочисленным и вещественным показателем, подумать на тему относительно того, как решить вопрос с возведением в степень — т. е. в степень можно возводить только квадратные матрицы, но не создавать же отдельный класс для квадратных и для прямоугольных матриц, или создавать?), произведение вектора и матрицы.
- Реализовать поэлементные операции для элементов векторов и матриц — арифметические операции со скалярной величиной, поэлементное возведение в степень.
- Провести сравнение скоростей обработки предложенного вами способа хранения и обработки разреженных матриц и выполнение тех же действий над векторами и матрицами, хранящимся с помощью стандартного контейнера `vector`.
- Результаты работы оформить в виде отчета или файла `readme` если код будет на `Git`.

Выполнение

Исходники проекта доступны по ссылке:

<https://github.com/buffer404/cpp-course>

Для разреженных матриц предложено иметь отдельный класс `expMatrix`, хранящий экземпляры структуры `nonZeroElem` и собственную размерность. Каждый экземпляр структуры хранит два поля: вектор ненулевых значений элемента матрицы, свою размерность и собственный

индекс в векторе вверх по уровню. То есть, по сути, матрица представляет собой вектор векторов с оберткой. Это позволяет создавать матрицы любой размерности.

Индексирование, транспонирование, умножение и прочие операции эксплуатируют `std::vector` для своей работы и стандартные итераторы.

Проверки условий матриц, “непопадание” в ненулевые элементы при индексировании обрабатываются с помощью инструментов библиотеки `algorithm` с помощью проверки, является ли хотя бы один ненулевой вектор в матрице подходящим по индексу (рекурсивно).

Поскольку поэлементные операции могут привести к значительному росту количества элементов в памяти, вместо этого матрица хранит поле “смещение от нуля”, которое обозначает собой значение, которое будет у всех нулевых элементов.

Реализованы все необходимые методы и операторы для разреженных матриц. Пример вывода после операций с такими матрицами. Обратная матрица:

```
matrix:
1      3      1
2      4      1
1      1      1

Determinant = -2
backwards matrix:
3      -2     -1
-1      0      1
-2      2     -2
```

Скалярные операции:

```

before adding 1:
1      3      1
2      4      1
1      1      1
added 1:
2      4      2
3      5      2
2      2      2
subtracted 2:
0      2      0
1      3      0
0      0      0
mul 2:
0      4      0
2      6      0
0      0      0
pow 2:
0      16     0
4      36     0
0      0      0

```

Также проведено сравнение скорости работы по сравнению с векторами из STL:

```

testing speeds of matrices...
test1: creating matrices:
Vector took      =    11082062[μs]
SMatrix took     =     238[μs]
filling matrices..
test2: multiplying by value
Vector took      =    14887036[μs]
SMatrix took     =    19416[μs]
test3: assign some value
Vector took      =     366[μs]
SMatrix took     =     81[μs]
test4: same but in the end of list
Vector took      =     276[μs]
SMatrix took     =     42[μs]
test5: access 1000 random cells
Vector took      =    29394[μs]
SMatrix took     =   1199090[μs]

```

Пример сравнения скоростей. В тесте создавались матрицы размером 1000x2000 элементов, в случайные ячейки плотностью примерно 1/1000 записывались случайные числа. После этого производился тест скалярного

умножения, присвоения значения ячейке, присвоения ячейке в конце списка, чтения из 1000 случайных ячеек.

Выводы

В ходе работы было получено значительное количество знаний и опыта работы с stl в C++. Был получен опыт решения прикладной задачи, а также применения методов ООП. В результате получен способ хранения разреженных матриц, более дешевый по памяти в сравнении с Vector библиотекой, однако более медленный при таких операциях, как поиск и индексирование.

В большинстве простых тестов разреженные матрицы оказались дешевле стандартных векторов, однако на больших количествах доступа проявляется усложненная индексация в спроектированной структуре. В качестве способа ускорения можно использовать хеш-таблицы для пар “ключ - значение”, дабы снизить скорость доступа. Дополнительное ускорение в арифметических операциях было получено за счет использования подвижного нуля.